VITyarthi | Open Source Software

The Open Source Audit

A Capstone Project for OSS NGMC Course

Course: Open Source Software | Unit Coverage: 1 – 5 Max Marks: 100

Student Name	Registration Number
RESHABH LAHOTI	**24BCE10343**
Slot	**Date of Submission**
E11	**31 March 2026**

• Introduction

Most of the software that powers the modern world — the servers that run websites, the tools that developers use every day, the operating systems inside supercomputers and smartphones
— was not built by a single company and sold for profit. It was built openly, shared freely, and improved collectively. That is the essence of open source.

This project asks you to stop and think about that.

You will choose one real open-source software project — something you can download, read about, and interact with on Linux — and conduct a structured audit of it. Not just 'what does it do,' but why it exists, who built it, what freedoms its license actually gives you, what philosophical values it represents, and how it has shaped the world around it.

Alongside the written report, you will write five shell scripts that demonstrate practical Linux skills. These are not meant to be impressive programs — they are meant to show that you understand how open-source tools work at the command line, and how automation connects to the philosophy of sharing and transparency.

Why this matters	Every line of code you write in your career will sit on top of open-source foundations. Understanding the philosophy behind that is not optional — it is part of being a professional.

• Choosing Your Software

Pick one open-source project from the list below, or propose your own (subject to instructor approval). Your choice will be the subject of your entire report and all five shell scripts.

Approved software options

Software	Category	License	Why it's interesting
Linux Kernel	Operating System	GPL v2	*The foundation everything else runs on*
Apache HTTP Server	Web Server	Apache 2.0	*Powers ~30% of all websites globally*
MySQL	Database	GPL v2 / Commerci al	*A license with a dual-model story to tell*
Firefox	Web Browser	MPL 2.0	*A nonprofit fighting for an open web*
VLC Media Player	Multimedia	LGPL / GPL	*Plays anything — built by students in Paris*
LibreOffice	Office Suite	MPL 2.0	*Born from a community fork — a real lesson*
Python	Programming Language	PSF License	*A language shaped entirely by community*
Git	Version Control	GPL v2	*The tool Linus built when proprietary failed him*

Your choice	Write your chosen software here before submission: ***Git*** _____

• Project Report Structure

Your report should be 12 to 16 pages long. Each section below maps to a unit in the course. Do not pad with screenshots — every page should contain original thinking and analysis written in your own words.

Part A — Origin and Philosophy (Units 1 & 2)

This is the most important section of the report. Allocate at least four pages to it.

A1. The problem this software was created to solve

Git was not created because developers wanted a better tool.
It was created because they suddenly had no tool they could trust.

In 2005, the Linux kernel project depended on a system called BitKeeper. It worked well technically, but it came with a hidden cost: control was not in the hands of the developers. When that relationship broke, access was revoked. Overnight, one of the largest open-source projects in the world lost its coordination system.

This is the real origin of Git—not convenience, but dependency failure.

Linus Torvalds did not try to replace BitKeeper with something similar. He rejected the model entirely. Instead of central control, he designed Git around a simple but powerful idea:

> Every developer should have a complete copy of the project.

This changed everything.

Earlier systems assumed a central authority. Git removed that assumption. Each contributor could work independently, verify history locally, and collaborate without relying on a single server. This was not just a technical improvement — it was a shift in power.

Git also prioritized:

- Speed
- Integrity
- Distributed workflow — no single point of failure

But the deeper reason Git succeeded is this: It aligned the tool with the philosophy of open source.

If the code is open, the process managing that code cannot be controlled by a closed system. Git solved not just a development problem, but a structural contradiction.

A2. The license — what it actually says

Git was released under the GNU General Public License version 2 (GPL v2). This license is one of the most important in the open-source world because it guarantees user freedoms while also enforcing certain responsibilities.

The Four Freedoms

- Defined by the GNU Project, the four freedoms are:
- Freedom to run the program for any purpose
- Freedom to study how the program works
- Freedom to modify the program
- Freedom to distribute copies (modified or not)

Can a Company Modify and Sell Git Without Sharing Changes?

No, not completely. Under GPL v2:

A company can sell a modified version of Git

But they must release the source code of their modification.

This is called a **copyleft** license.

It prevents companies from taking open-source work, modifying it, and turning it into closed software.

Feature	GPL	MIT
Code must remain open	Yes	No
Commercial use	Allowed	Allowed
Modification sharing	Required	Optional

If I create my own software, I would choose GPL if I want to ensure openness and MIT if I want maximum adoption, even in proprietary systems.

Free as in “Free Beer” vs “Free Speech”

- Free beer → no cost
- Free speech → freedom to modify and share

Git is free in both senses:

- You can download it without paying
- You can modify and redistribute it freely

A3. The ethics of open source

Open source is often presented as an obvious good.
In reality, it is full of trade-offs.

Should All Software Be Open Source?

Argument For

Open source increases:

- Transparency → anyone can inspect the code
- Security → more eyes can find flaws
- Collaboration → faster innovation

Argument Against

Not all software benefits from being open.

- Some systems (e.g., security tools) may require controlled access
- Companies need incentives to invest in complex products
- Open projects sometimes lack accountability and structured support

Complete openness can create **diffused responsibility**, where no one is fully accountable.

Is It Ethical for Companies to Profit from Open Source?

This is not a simple yes or no.

Companies like Red Hat or Google build products using open-source foundations.

So the real question is not:

- “Is profit wrong?”

It is:

“Is the exchange fair?”

“Standing on the Shoulders of Giants”

In software, this phrase is literal.

Git itself builds on:

- earlier version control ideas
- Unix philosophy
- collaborative development practices

Modern tools then build on Git.

This creates a layered system of innovation where:

- No one starts from zero
- Progress compounds over time

Open source accelerates this process.

But there is a subtle tension:

- If everyone builds on existing work, is anything truly original?

The answer is that originality shifts from invention to **integration and improvement**.

Open source does not remove control. It **redistributes it**.

Instead of one company deciding everything, control is spread across:

- contributors
- maintainers
- communities

Git represents more than a tool for version control.
It represents a response to a failure of trust, and a deliberate move toward systems where:

- dependency is minimized
- transparency is enforced
- control is shared

Part B — Linux Footprint (Unit 2)

Installation Method

Git is installed using:

```bash
sudo apt install git
```

This shows how open-source tools are distributed through package managers.

Key Directories

After installation:

- **Main Executable:** /usr/bin/git
- **System-Wide Configuration:** /etc/gitconfig
- **User Configuration:** ~/.gitconfig
- **Logs (if applicable):** /var/log
-

User and Group

Git operates as the **current user**, not with root privileges, which is significant because this approach:

- Mitigates security vulnerabilities.
- Aids in preventing damage that could affect the entire system.

Service Management

```bash
git --version
git init
```

Git functions as a command-line utility, not a persistent background service like a web server. For instance:

Update Model

Updates come via:

```bash
sudo apt update
sudo apt upgrade
```

Security patches are contributed by the global community and distributed.

Part C — The FOSS Ecosystem (Units 3 & 4)

Dependencies for Git:

Git relies on the following core components to function:

- The C programming language
- Standard software libraries
- Essential Shell utilities

Impact and Capabilities Enabled by Git:

Git has fundamentally transformed modern software development by enabling:

- Platforms like GitHub and GitLab
- Widespread open-source collaboration

The current state of modern development workflows is directly dependent on the existence of Git.

Role of Git in the Development Stack

Git, while not an intrinsic component of the LAMP stack, plays a crucial supportive role by enabling:

- Code deployment and management
- Comprehensive version tracking
- Effective developer collaboration

The Git Community and Governance

The Git project is characterized by an open community structure:

- It utilizes mailing lists for communication.
- It benefits from contributions by a global network of developers.
- It is maintained by a group of experienced individuals, with Linus Torvalds providing leadership.

Governance is open, but guidance from seasoned maintainers ensures stability and direction.-----**Part D – Open Source (Git) vs. Proprietary Alternatives (Critical Analysis)**

A comparison of Git against a proprietary version control system highlights key differences, emphasizing that the choice depends on specific needs.

Dimension	**Open Source (Git)**	**Proprietary Alternative**
Cost	Completely free to use, modify, and redistribute.	Typically involves mandatory licensing fees or subscription costs.
Security & Auditing	Source code is publicly available, allowing for transparent auditing by anyone.	Source code is closed; users must rely entirely on the vendor's security assurances.

Support & Reliability	Relies on community-driven support via forums, documentation, and a network of contributors.	Provides dedicated, professional support often backed by formal Service-Level Agreements (SLAs).
Freedom to Modify	Highly flexible and customizable to fit diverse workflows and needs.	Limited in customization, subject to the restrictions imposed by the controlling company.
Control	Governed by a decentralized global community without a single corporate owner.	Controlled by a single company with full authority over all features and developmental changes.

Overall Verdict	**Open Source (Git)**	**Proprietary Alternative**
	Git offers superior accessibility, transparency, flexibility, and user control. Its open nature and robust community generally	Proprietary tools can be valuable in enterprise settings that mandate guaranteed, structured support, but they inherently limit user

	make it the superior choice for most real-world applications.	freedom and transparency.

A major benefit of Git is its flexible, distributed architecture, which allows users to work independently while maintaining complete, local copies of the project repository. This design ensures high reliability and scalability for projects of all sizes. While proprietary solutions may offer specialized enterprise features and guaranteed support, they often come at the cost of limited user control and customization. Overall, Git provides a more empowering and well-rounded solution, merging technical efficiency with the core tenets of open-source development. Therefore, Git is highly recommended for any real-world deployment, not only for its technical capabilities but also for the invaluable opportunity it provides to engage with a collaborative, global ecosystem of innovation.

Part D — Open Source vs Proprietary (Critical Analysis)

Compare your chosen software against its closest proprietary alternative. Be specific and honest — do not assume open source is always better.

Dimension	Open Source (Git)	Proprietary Alternative
Cost	Completely free to use, modify, and distribute	Often requires licensing fees or subscriptions
Security (who can audit the code?)	Source code is publicly available and can be audited by anyone	Source code is closed; users must trust the company
Support and reliability	Community-driven support through forums, documentation, and contributors	Dedicated professional support with service-level agreements
Freedom to modify	Highly customizable and adaptable to different workflows	Limited customization based on company restrictions

Community vs corporate control	Controlled by a global community, no single owner	Controlled by a company with full authority over features and changes
Your overall verdict	Git is more accessible, transparent, flexible, and gives users full control. While proprietary tools may offer structured support, Git's open nature and strong community make it the better overall choice for most real-world scenarios.	Proprietary tools can be useful in enterprise environments requiring guaranteed support, but they limit user freedom and transparency.

Git is one of the most influential open-source tools in modern software development, showing how a community-driven approach can produce software that is both powerful and widely trusted.
Developed by Linus Torvalds, Git emerged out of necessity but quickly evolved into a global standard for version control. Its transparency is one of its greatest strengths, as the open availability of its source code allows developers to inspect, modify, and improve it freely. This openness builds trust and enables faster identification and resolution of security issues, unlike proprietary tools that rely on closed systems and company-controlled updates.

Another key advantage of Git is its flexibility and distributed architecture, which allows users to work independently while maintaining complete copies of project repositories. This makes it highly reliable and suitable for both small and large-scale development. While proprietary tools may offer structured support and enterprise features, they often limit user control and customization. Overall, Git provides a more balanced and empowering solution by combining technical efficiency with the core values of open-source development. It is highly recommended for real-world use, not only for its capabilities but also for the opportunity it offers to participate in a collaborative and innovative global ecosystem.

Shell Script Tasks

You will write five shell scripts. Each script must be submitted as a plain .sh file and documented inside your report with: the script code, a screenshot of it running, and a paragraph explaining what it does and which shell scripting concepts it uses.

Script 1 — System Identity Report (Unit 1 & 2)

Write a script that introduces the Linux system like a welcome screen. It should display:

- The Linux distribution name and kernel version
- The current logged-in user and their home directory
- System uptime and current date/time
- A message stating which open-source license covers the OS

Concepts to use: variables, echo, command substitution ($()), basic output formatting.

script1_system_identity.sh ● script2_package_inspector.sh ●

script1_system_identity.sh

```bash
#!/bin/bash
# ========================================
# Script 1: System Identity Report
# Author: RESHABH LAHOTI | 24BCE10343
# Description: Displays system information
# ========================================
# --- Variables ---
STUDENT_NAME="= Reshabh Lahoti"
SOFTWARE_CHOICE="Git"
LICENSE="GPL v2"
# --- System Info ---
KERNEL=$(uname -r)
USER_NAME=$(whoami)
HOME_DIR=$HOME
UPTIME=$(uptime | awk -F'(up |,)' '{print $2}')
CURRENT_DATE=$(date "+%d %B %Y %H:%M:%S")
# Works for both Linux and Mac
DISTRO=$(uname -s)
# --- Display Output ---
echo "========================================"
echo " Open Source Audit — $STUDENT_NAME"
echo "========================================"
echo ""
echo "Chosen Software: $SOFTWARE_CHOICE"
echo ""
echo "System Information"
echo "----------------------------------------"
echo "OS           : $DISTRO"
echo "Kernel       : $KERNEL"
echo "Logged in as : $USER_NAME"
echo "Home Dir     : $HOME_DIR"
echo "Uptime       : $UPTIME"
echo "Date & Time  : $CURRENT_DATE"
echo ""
echo "License Info"
echo "----------------------------------------"
echo "This system runs Unix/Linux based OS."
echo "Git is licensed under $LICENSE."
echo "You are free to use, modify, and share it."
echo ""
echo ""“In open source, you control your own destiny.”""
echo "========================================"
```

Output

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mac@Devs-MacBook-Air-2 24bce10343 % chmod +x script1_system_identity.sh
mac@Devs-MacBook-Air-2 24bce10343 % ./script1_system_identity.sh
========================================
 Open Source Audit - = Reshabh Lahoti
========================================

Chosen Software: Git

System Information
----------------------------------------
OS           : Darwin
Kernel       : 25.0.0
Logged in as : mac
Home Dir     : /Users/mac
Uptime       :  2:23
Date & Time  : 31 March 2026 22:53:11

License Info
----------------------------------------
This system runs Unix/Linux based OS.
Git is licensed under GPL v2.
You are free to use, modify, and share it.

"In open source, you control your own destiny."
========================================
mac@Devs-MacBook-Air-2 24bce10343 %
```

Script 2 – FOSS Package Inspector (Unit 2)

Write a script that checks whether your chosen software is installed on the system, finds its version, and uses a case statement to print a short description of its purpose based on the package name.

Concepts to use: if-then-else, case statement, rpm -qi or dpkg -l, pipe with grep.

script1_system_identity.sh ● script2_package_inspector.sh ●

script2_package_inspector.sh

```bash
#!/bin/bash
# ========================================
# Script 2: FOSS Package Inspector
# Author: Reshabh Lahoti | 24BCE10343
# Description: Checks if a package is installed
# and displays details + philosophy
# ========================================
# --- Package Name ---
PACKAGE="git"
echo "========================================"
echo " FOSS Package Inspector"
echo "========================================"
echo ""
# --- Check if package is installed ---
if command -v $PACKAGE &> /dev/null
then
    echo "[✓] $PACKAGE is installed on this system."
    echo ""
    echo "Version Information:"
    echo "----------------------------------------"
    $PACKAGE --version
else
    echo "[✗] $PACKAGE is NOT installed."
    echo "Install using:"
    echo "brew install $PACKAGE   (Mac)"
    echo "sudo apt install $PACKAGE   (Linux)"
fi
echo ""
echo "Philosophy Note:"
echo "----------------------------------------"
# --- Case Statement ---
case $PACKAGE in
    git)
        echo "Git: Created to remove dependency on proprietary systems."
        echo "It represents control, transparency, and distributed development."
        ;;
    apache2|httpd)
        echo "Apache: A web server that helped build the open internet."
        ;;
    firefox)
        echo "Firefox: A browser focused on privacy and open web standards."
        ;;
    python3|python)
        echo "Python: A community-driven language promoting simplicity and collaboration."
        ;;
    vlc)
        echo "VLC: A media player built openly to support all formats without restriction."
        ;;
    *)
        echo "$PACKAGE: An open-source tool contributing to modern computing."
        ;;
esac
echo ""
echo "========================================"
```

Script 3 — Disk and Permission Auditor (Unit 2)

Write a script that loops through a list of important system directories and reports: how much space each uses, and the owner and permissions of the directory. Use a for loop.

Concepts to use: for loop, df, ls -ld, awk or cut to extract fields.

script1_system_identity.sh | script2_package_inspector.sh | script3_disk_audit.sh

script3_disk_audit.sh

```bash
#!/bin/bash

# ========================================
# Script 3: Disk and Permission Auditor
# Author: RESHABH LAHOTI | 24BCE10343
# Description: Checks directory size and permissions
# ========================================

# --- Directories to check ---
DIRS=("/etc" "/var/log" "/home" "/usr/bin" "/tmp")

echo "========================================"
echo " Disk and Permission Audit"
echo "========================================"
echo ""

printf "%-15s %-25s %-10s\n" "Directory" "Permissions (Owner Group)" "Size"
echo "----------------------------------------------------------------"

# --- Loop through directories ---
for DIR in "${DIRS[@]}"
do
    if [ -d "$DIR" ]; then

        # Get permissions, owner, group
        PERMS=$(ls -ld "$DIR" | awk '{print $1, $3, $4}')

        # Get size
        SIZE=$(du -sh "$DIR" 2>/dev/null | cut -f1)

        # Print formatted output
        printf "%-15s %-25s %-10s\n" "$DIR" "$PERMS" "$SIZE"

    else
        printf "%-15s %-25s %-10s\n" "$DIR" "Not Found" "-"
    fi
done

echo ""
echo "----------------------------------------"
echo " Git-Specific Check"
echo "----------------------------------------"

# --- Check Git config ---
GIT_CONFIG="$HOME/.gitconfig"

if [ -f "$GIT_CONFIG" ]; then
    PERMS=$(ls -l "$GIT_CONFIG" | awk '{print $1, $3, $4}')
    SIZE=$(du -h "$GIT_CONFIG" 2>/dev/null | cut -f1)

    echo "[✔] Git config found: $GIT_CONFIG"
    echo "Permissions: $PERMS"
    echo "Size: $SIZE"
else
    echo "[✘] No Git config found."
    echo "Run: git config --global user.name \"Your Name\""
fi

echo ""
echo "========================================"
```

Output :

```
mac@Devs-MacBook-Air-2 24bce10343 % chmod +x script3_disk_audit.sh
mac@Devs-MacBook-Air-2 24bce10343 % ./script3_disk_audit.sh
========================================
 Disk and Permission Audit
========================================

Directory       Permissions (Owner Group) Size
----------------------------------------------------------------
/etc            lrwxr-xr-x@ root wheel      0B
/var/log        drwxr-xr-x root wheel      50M
/home           lrwxr-xr-x root wheel       0B
/usr/bin        drwxr-xr-x root wheel      80M
/tmp            lrwxr-xr-x@ root wheel      0B

----------------------------------------
 Git-Specific Check
----------------------------------------
[✗] No Git config found.
Run: git config --global user.name "Your Name"

========================================
mac@Devs-MacBook-Air-2 24bce10343 %
```

Script 4 — Log File Analyzer (Units 2 & 5)

Write a script that reads a log file line by line, counts how many lines contain a keyword (like ERROR or WARNING), and prints a summary. Use a while-read loop and an if-then inside it.

Concepts to use: while read loop, if-then, counter variables, command-line arguments ($1).

Starter structure

script1_system_identity.sh | script2_package_inspector.sh | script3_disk_audit.sh | script4_log_analyzer.sh

script4_log_analyzer.sh

```bash
#!/bin/bash

# ========================================
# Script 4: Log File Analyzer
# Author: RESHABH LAHOTI | 24BCE10343
# Description: Counts keyword occurrences in log file
# ========================================

# --- Input Arguments ---
LOGFILE=$1
KEYWORD=${2:-"error"}   # default keyword = error

COUNT=0

echo "========================================"
echo " Log File Analyzer"
echo "========================================"
echo ""

# --- Check if file exists ---
if [ ! -f "$LOGFILE" ]; then
    echo "[✘] File not found: $LOGFILE"
    echo "Usage: ./script4_log_analyzer.sh <logfile> [keyword]"
    exit 1
fi

# --- Check if file is empty ---
if [ ! -s "$LOGFILE" ]; then
    echo "[!] File is empty: $LOGFILE"
    exit 0
fi

echo "Analyzing file: $LOGFILE"
echo "Keyword       : $KEYWORD"
echo ""

# --- Read file line by line ---
while IFS= read -r LINE
do
    if echo "$LINE" | grep -iq "$KEYWORD"; then
        COUNT=$((COUNT + 1))
    fi
done < "$LOGFILE"

# --- Print result ---
echo "----------------------------------------"
echo "Keyword '$KEYWORD' found $COUNT time(s)."

# --- Show last 5 matches ---
echo ""
echo "Last 5 matching lines:"
grep -i "$KEYWORD" "$LOGFILE" | tail -5

echo ""
echo "========================================"
```

PROBLEMS | OUTPUT | DEBUG CONSOLE | TERMINAL | PORTS

```
mac@Devs-MacBook-Air-2 24bce10343 % chmod +x script4_log_analyzer.sh
mac@Devs-MacBook-Air-2 24bce10343 % nano sample.log
mac@Devs-MacBook-Air-2 24bce10343 % ./script4_log_analyzer.sh sample.log error
========================================
 Log File Analyzer
========================================

Analyzing file: sample.log
Keyword       : error

----------------------------------------
Keyword 'error' found 3 time(s).

Last 5 matching lines:
ERROR: Failed to load module
error: Disk issue detected
ERROR: Timeout occurred

========================================
mac@Devs-MacBook-Air-2 24bce10343 %
```

Script 5 — The Open Source Manifesto Generator (Unit 5)

This is the most creative script. Write a script that generates a personalized open source philosophy statement by asking the user three questions interactively, then composing a short paragraph using their answers and saving it to a .txt file.

Concepts to use: read for user input, string concatenation, writing to a file with >, date command, aliases concept demonstrated via a comment.

script1_system_identity.sh | script2_package_inspector.sh | script3_disk_audit.sh | script4_log_analyzer.sh | script5_manifesto.sh

script5_manifesto.sh

```bash
#!/bin/bash

# ========================================
# Script 5: Open Source Manifesto Generator
# Author: RESHABH LAHOTI | 24BCE10343
# Description: Generates a personalized manifesto
# ========================================

echo "========================================"
echo " Open Source Manifesto Generator"
echo "========================================"
echo ""

# --- User Input ---
read -p "1. Name one open-source tool you use daily: " TOOL
read -p "2. In one word, what does 'freedom' mean to you? " FREEDOM
read -p "3. What would you build and share freely? " BUILD

# --- Date and File ---
DATE=$(date "+%d %B %Y")
OUTPUT="manifesto_$(whoami).txt"

echo ""
echo "Generating your manifesto..."
echo ""

# --- Write to File ---
echo "========================================" > "$OUTPUT"
echo " MY OPEN SOURCE MANIFESTO" >> "$OUTPUT"
echo " Generated on: $DATE" >> "$OUTPUT"
echo " By: $(whoami)" >> "$OUTPUT"
echo "========================================" >> "$OUTPUT"
echo "" >> "$OUTPUT"

echo "I believe in the power of open-source software." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Every day, I rely on $TOOL — not built for profit alone," >> "$OUTPUT"
echo "but shaped by a community that values shared progress." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "To me, freedom means $FREEDOM." >> "$OUTPUT"
echo "It is the ability to learn, modify, and build without restriction." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "One day, I will create $BUILD and share it openly," >> "$OUTPUT"
echo "so others can build on it, just as I have built on the work of others." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "Open source is not just code — it is collaboration." >> "$OUTPUT"
echo "" >> "$OUTPUT"
echo "========================================" >> "$OUTPUT"

# --- Display Output ---
echo "[✓] Manifesto saved to: $OUTPUT"
echo ""
echo "Your Manifesto:"
echo "----------------------------------------"
cat "$OUTPUT"

echo ""
echo "========================================"
```

Output :

```
PROBLEMS    OUTPUT    DEBUG CONSOLE    TERMINAL    PORTS

mac@Devs-MacBook-Air-2 24bce10343 % chmod +x script5_manifesto.sh
mac@Devs-MacBook-Air-2 24bce10343 % ./script5_manifesto.sh
========================================
 Open Source Manifesto Generator
========================================

1. Name one open-source tool you use daily: Git
freedom
a startup platform2. In one word, what does 'freedom' mean to you? 3. What would you build and share freely?

Generating your manifesto...

[✔] Manifesto saved to: manifesto_mac.txt

Your Manifesto:
----------------------------------------
========================================
 MY OPEN SOURCE MANIFESTO
 Generated on: 31 March 2026
 By: mac
========================================

I believe in the power of open-source software.

Every day, I rely on Git — not built for profit alone,
but shaped by a community that values shared progress.

To me, freedom means freedom.
It is the ability to learn, modify, and build without restriction.

One day, I will create a startup platform and share it openly,
so others can build on it, just as I have built on the work of others.

Open source is not just code — it is collaboration.

========================================

========================================
mac@Devs-MacBook-Air-2 24bce10343 %
```

2. Project Timeline

This timeline is a guide, not a rigid rule. Some students will move faster on the technical parts; others will spend more time on the philosophical sections. Adjust based on your strengths — but do not leave the scripts to the final stretch.

Day	Focus	Tasks	Deliverable
1	**Setup**	Choose software. Research its origin story. Read the license text. Set up your Linux environment (VM or lab system). Install your chosen software.	*Choice confirmed*
2	**Philosophy**	Write Part A1 — the origin story. Take notes on what problem the software solved and for whom.	*Draft A1*
3	**License & Ethics**	Write Part A2 — license analysis. Write Part A3 — ethical reflection. These sections need your own thinking, not summaries.	*Draft A2 + A3*
4	**Linux Footprint**	Install software on Linux. Document directories, user, permissions. Write Part B.	*Draft Part B + screenshots*
5	**Scripts 1 & 2**	Write and test Script 1 (System Identity) and Script 2 (Package Inspector). Add them to your report with explanations.	*Scripts 1–2 working*
6	**Scripts 3 & 4**	Write and test Script 3 (Disk Auditor) and Script 4 (Log Analyzer). Test with a real log file.	*Scripts 3–4 working*
7	**Ecosystem**	Write Part C — FOSS ecosystem map. Research dependencies and community.	*Draft Part C*
8	**Comparison & Script 5**	Write Part D — comparison table and verdict. Write and test Script 5 (Manifesto Generator).	*Draft Part D + Script 5*
9	**Report polish**	Compile full report. Write introduction and conclusion. Add screenshots. Check page count (12–16 pages).	*Complete draft*
10	**Submission**	Final review. Submit report + 5 .sh files. Be ready to explain any script in a short viva if asked.	*Final submission*

3. Evaluation Rubric

Total marks: 100. The report carries 60 marks and the shell scripts carry 40 marks. There is no viva unless the evaluator suspects the work is not original — in which case any student may be called for a brief oral explanation.

Report (60 marks)

#	Component	Full marks	Passing (50%)	Notes
1	Origin story — depth, original research, storytelling	12	6	*Part A1*
2	License analysis — accuracy and understanding of freedoms	12	6	*Part A2*
3	Ethical reflection — quality of argument and original thinking	10	5	*Part A3*
4	Linux footprint — technical accuracy and screenshots	10	5	*Part B*
5	Ecosystem and LAMP connection	8	4	*Part C*
6	Open source vs proprietary comparison	8	4	*Part D*

Shell Scripts (40 marks — 8 marks each)

Each script is marked on three criteria:

Criterion	Marks	What we look for
Correctness — does it run without errors?	4	*Script executes on Linux and produces expected output*
Concepts — does it use the right shell constructs?	2	*Loops, if-then, case, variables used appropriately for the task*
Comments and documentation	2	*Every non-obvious line has a comment; script purpose is clear*

Academic integrity	All written sections must be in your own words. Code that is copy-pasted from the internet without understanding will be zero-marked. Running a script and explaining what it does in a conversation is considered a passing demonstration of understanding.

4. Reference Resources

Use these as starting points, not as sources to copy from. The report must be in your own words.

On open source philosophy

- GNU Project — 'The Free Software Definition': gnu.org/philosophy/free-sw.html
- Open Source Initiative — 'The Open Source Definition': opensource.org/osd
- Eric S. Raymond — 'The Cathedral and the Bazaar': catb.org/~esr/writings/cathedral-bazaar
- Linus Torvalds — 'Just for Fun' (autobiography, available at most libraries)

On Linux and shell scripting

- The Linux Command Line by William Shotts — freely available at linuxcommand.org
- GNU Bash Manual: gnu.org/software/bash/manual
- Linux man pages: man.cx (online) or type 'man <command>' in your terminal

For finding license texts

- SPDX License List: spdx.org/licenses
- Choose a License: choosealicense.com
- Your software's official repository on GitHub or GitLab — the LICENSE file is always there

A closing thought

Every tool you will use in your career — the editor, the compiler, the server, the database — was shaped by people who chose to build in the open and share their work freely. Understanding why they made that choice, and what it means for the world, is not a footnote to your technical education. It is the foundation of it.

5. Submission Requirements

All submissions are made through the VITyarthi portal. You must submit three things: a public GitHub repository link, a README.md inside the repository, and a project report PDF. All three are compulsory. Incomplete submissions will not be evaluated.

What to submit on the VITyarthi portal

Item	Details	Compulsory?

GitHub repo link	The repository must be public. It must contain all five .sh script files and a README.md. Name the repo: oss-audit-[rollnumber]	**Yes**
README.md	Must be inside the GitHub repository. Must include: student name and roll number, chosen software, description of each script, step-by-step instructions to run each script on Linux, and any dependencies required.	**Yes**

Project report PDF	Upload directly on the VITyarthi portal. Must follow the structure in Section 3. Must be 12 to 16 pages. Must be your own writing.	**Yes**

On the VITyarthi portal submission form, paste the GitHub repository URL, confirm the README is present in the repository, and upload the project report PDF. All three must be submitted together in a single submission — partial submissions will not be accepted.

Evaluation pipeline

Every submission goes through an automated evaluation pipeline before it reaches an evaluator. Be aware of the following checks that run on every submitted report and script set.

Check	What it detects	Consequence
Plagiarism detection	Report text copied from websites, Wikipedia, other students' submissions, or any published source without attribution. Cross-submission similarity between enrolled students is also flagged.	Zero on the full report. Referred to the disciplinary committee if similarity exceeds 40%.
AI generation detection	Report sections generated by AI tools such as ChatGPT, Gemini, Claude, or similar. The pipeline checks for linguistic patterns, sentence uniformity, and structural signatures common to AI-generated text.	Flagged submissions are called for a mandatory viva. If the student cannot explain the content, zero marks are awarded.

A note on AI tools: you may use AI to understand concepts, debug scripts, or get explanations. You may not use AI to write your report. The distinction is the same as using a calculator versus asking someone else to sit your exam. The report must reflect your thinking, your words, and your conclusions.